

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I J.MOHAMED ASEEM(192511258) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

J.MOHAMED ASEEM

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program compiles successfully but produces wrong results and crashes at run time. It performs dynamic memory allocation, summation and a binary search. Analyze the logical and runtime errors in the program.

```
#include <stdio.h>
#include <stdlib.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid;
        else
            high = mid;
    }
    return -1;
}

int main() {
    int *list = (int*) malloc(5 * sizeof(int));
    int i, sum;
    for(i = 0; i < 5; i++)
        list[i] = i * 10;
    for(i = 0; i <= 5; i++)
        sum += list[i];
    int pos = binarySearch(list, 5, 50);
    printf("Found at %d\n", pos);
    free(list);
    printf("%d\n", list[0]);
    int x = 10;
    if(x = 20)
        printf("x is twenty\n");
    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all logical and runtime errors
2. Mention the line number and describe the nature of each error

(b) Classification of Errors (Understanding + Analysis)

1. Separate the errors into Logical Errors and Runtime Errors
2. Justify why none of them is reported as a compile-time error

(c) Trace and Evaluation (Evaluation Level)

1. Trace `binarySearch(list, 5, 50)` for the array `{0, 10, 20, 30, 40}` and show the values of `low`, `high` and `mid` in each iteration
2. Explain why the loop never terminates and what the correct updates should be

(d) Code Correction (Application Level)

1. Rewrite the corrected program
2. Ensure that memory is accessed only while it is valid
3. Ensure the search terminates for keys that are absent

(e) Impact Analysis (Evaluation Level)

1. Explain the effect of using the uninitialized variable `sum`
2. Explain the effect of use-after-free
3. Explain the effect of using `=` instead of `==` in a condition

(f) Advanced Compiler Perspective (Create Level)

1. Suggest compiler warning options and static-analysis checks that can detect these errors
2. Describe how runtime instrumentation (bounds checking, AddressSanitizer, Valgrind) can detect them early

Q2. The following LEX specification is written to recognize identifiers, keywords, integers, floating-point numbers and operators. The specification contains several errors. Analyze it carefully.

```
%{
#include <stdio.h>
}%

digit [0-9]
letter [a-zA-Z]

%%
{letter}({letter}|{digit})* { printf("IDENTIFIER\n"); }
"if"|"else"|"while"      { printf("KEYWORD\n"); }
{digit}+                  { printf("INTEGER\n"); }
{digit}+. {digit}+        { printf("FLOAT\n"); }
"="|"+"|"-"|"*"|"/"      { printf("OPERATOR\n"); }
[ \t\n]                   { }
.                          { printf("UNKNOWN\n"); }
```

```
int main() {
    yylex();
    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all errors in the LEX specification
2. Explain the effect of each error on the generated scanner

(b) Rule Matching (Understanding + Analysis)

1. Explain the two LEX conflict-resolution rules: longest match and first rule listed
2. Using these rules, explain why the input `if` is not recognized as `KEYWORD` by the given specification

(c) Token Recognition (Evaluation Level)

For the input: `if x1 = 12a4 + 3.14`

1. Show the output produced by the given specification
2. Show the output produced by the corrected specification
3. Explain why `12a4` behaves differently in the two cases

(d) Code Correction (Application Level)

1. Rewrite the corrected LEX specification
2. Add the missing section separator and the `yywrap` handling

(e) Extension (Application Level)

1. Add rules to recognize the relational operators `<`, `<=`, `==`, `!=`, `>`, `>=`
2. Add rules for a string literal and a single-line comment `//...`
3. Allow the underscore character in identifiers

(f) Advanced Thinking (Create Level)

1. Explain how LEX converts the regular expressions into a DFA and generates `lex.yy.c`
2. Write the commands to compile and run the scanner
3. Suggest how to report an error with its line number using `yylineno`

Q3. The following program fragment is passed through a lexical analyzer. It contains preprocessor directives, comments, keywords, operators and constants, and one lexical error. Analyze how the scanner processes the program.

```
#include <stdio.h>
#define LIMIT 100

int main() {
    int total = 0, i;
    float rate = 2.5e+;
    /* sum of first n numbers */
    for(i = 1; i <= LIMIT; i++) {
        total += i; // accumulate
```

```

    }
    if(total != 5050 && rate > 2) {
        printf("Sum = %d\n", total);
    }
    return 0;
}

```

Tasks:

(a) Token Identification (Analysis Level)

1. List the lexemes of the statement `for(i = 1; i <= LIMIT; i++) {` and give the token class of each (keyword, identifier, constant, operator, punctuation)
2. Count the total number of tokens in that statement
3. Count the tokens in the statement `total += i;`

(b) Lexical Error Analysis (Analysis Level)

1. Identify the lexical error and its line number
2. Explain which rule of the numeric constant is violated and suggest a valid replacement

(c) Scanner Behavior (Understanding Level)

1. State which parts of the program (comments, whitespace, preprocessor lines) are discarded and which are passed to the parser
2. Explain how the scanner distinguishes `+`, `++` and `+=` and also `<`, `<=` using longest match

(d) Symbol Table (Application Level)

1. Construct the symbol table entries for the identifiers `total`, `i`, `rate` and `LIMIT` with attributes: name, token class, data type, scope
2. Explain which phase fills in the data type attribute

(e) Regular Expressions (Application Level)

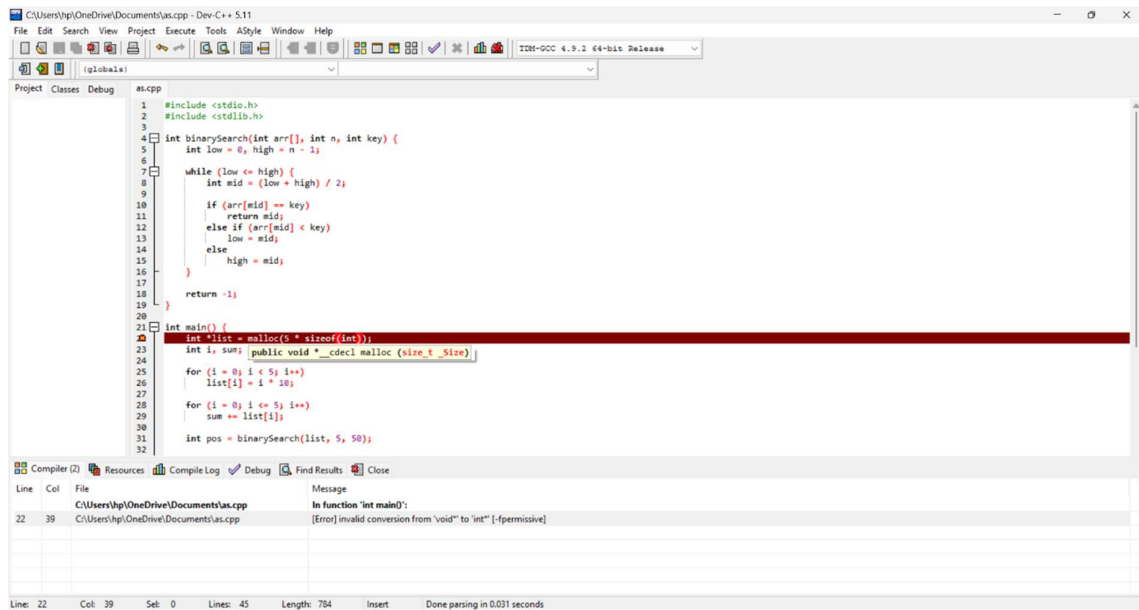
1. Write regular definitions for: integer constant, floating-point constant with exponent, identifier and relational operators
2. Show how the definitions accept `5050` and reject `2.5e+`

(f) Advanced Task (Create Level)

1. Explain the two-buffer input scheme with sentinels and how it supports look-ahead
2. Suggest improvements to the scanner for better error recovery and error messages

Answers

1.)Error code:



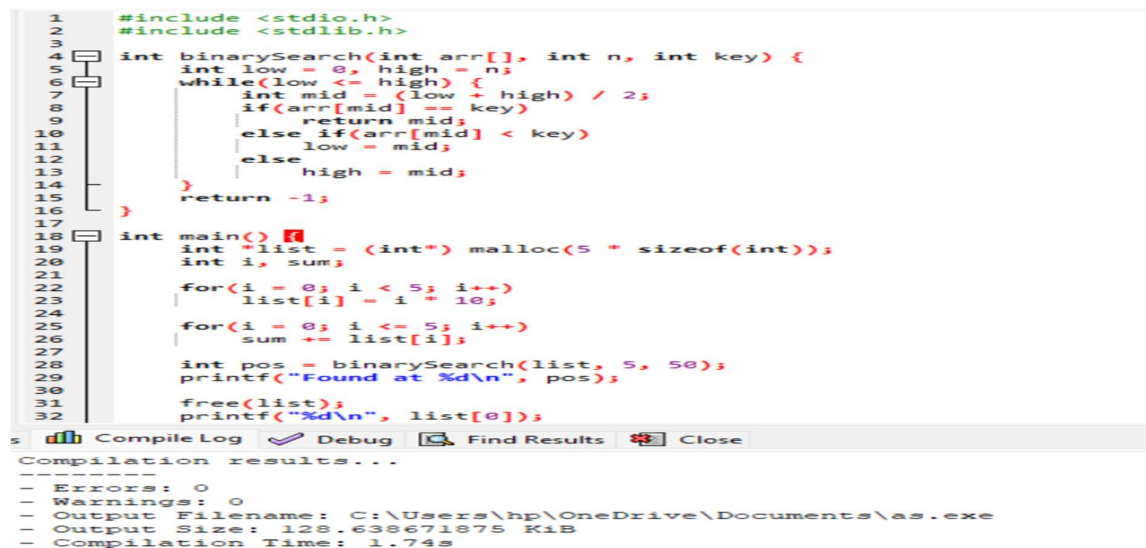
```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int binarySearch(int arr[], int n, int key) {
5     int low = 0, high = n - 1;
6
7     while (low <= high) {
8         int mid = (low + high) / 2;
9
10        if (arr[mid] == key)
11            return mid;
12        else if (arr[mid] < key)
13            low = mid;
14        else
15            high = mid;
16    }
17    return -1;
18 }
19
20 int main() {
21     int *list = malloc(5 * sizeof(int));
22     int i, sum; public void * _cdecl malloc (size_t _Size)
23
24     for (i = 0; i < 5; i++)
25         list[i] = i * 10;
26
27     for (i = 0; i <= 5; i++)
28         sum += list[i];
29
30     int pos = binarySearch(list, 5, 50);
31
32 }
```

Compiler (2) Resources Compile Log Debug Find Results Close

Line	Col	File	Message
22	39	C:\Users\hp\OneDrive\Documents\as.cpp	In function 'int main()':
		C:\Users\hp\OneDrive\Documents\as.cpp	[Error] invalid conversion from 'void' to 'int' [-fpermissive]

Line: 22 Col: 39 Sel: 0 Lines: 45 Length: 784 Insert Done parsing in 0.031 seconds

Corrected code:



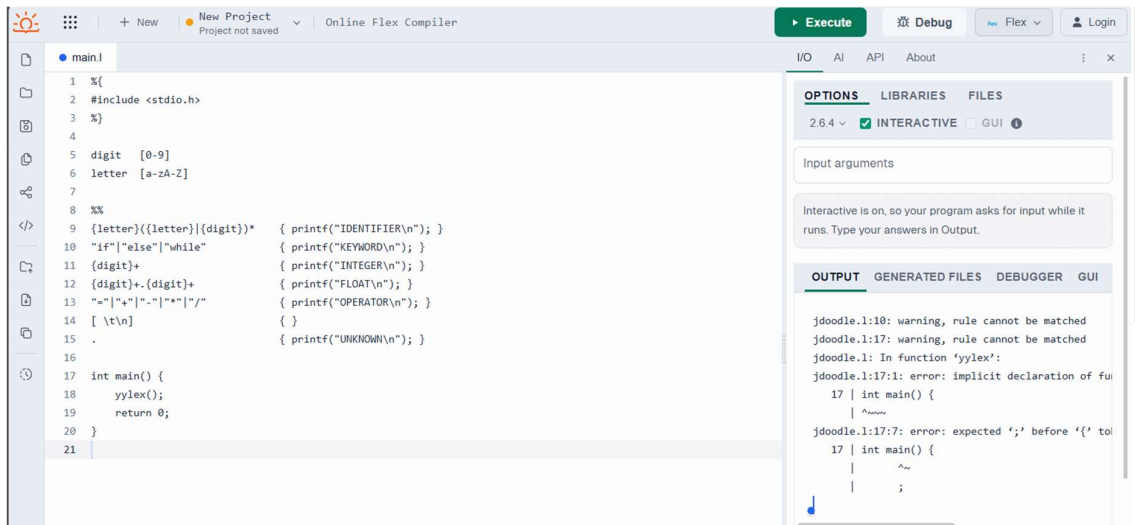
```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int binarySearch(int arr[], int n, int key) {
5     int low = 0, high = n;
6     while (low <= high) {
7         int mid = (low + high) / 2;
8         if (arr[mid] == key)
9             return mid;
10        else if (arr[mid] < key)
11            low = mid;
12        else
13            high = mid;
14    }
15    return -1;
16 }
17
18 int main() {
19     int *list = (int*) malloc(5 * sizeof(int));
20     int i, sum;
21
22     for (i = 0; i < 5; i++)
23         list[i] = i * 10;
24
25     for (i = 0; i <= 5; i++)
26         sum += list[i];
27
28     int pos = binarySearch(list, 5, 50);
29     printf("Found at %d\n", pos);
30
31     free(list);
32     printf("%d\n", list[0]);
33 }
```

Compile Log Debug Find Results Close

Compilation results...

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\hp\OneDrive\Documents\as.exe
- Output Size: 128.638671875 KiB
- Compilation Time: 1.74s

2.)Error Code:



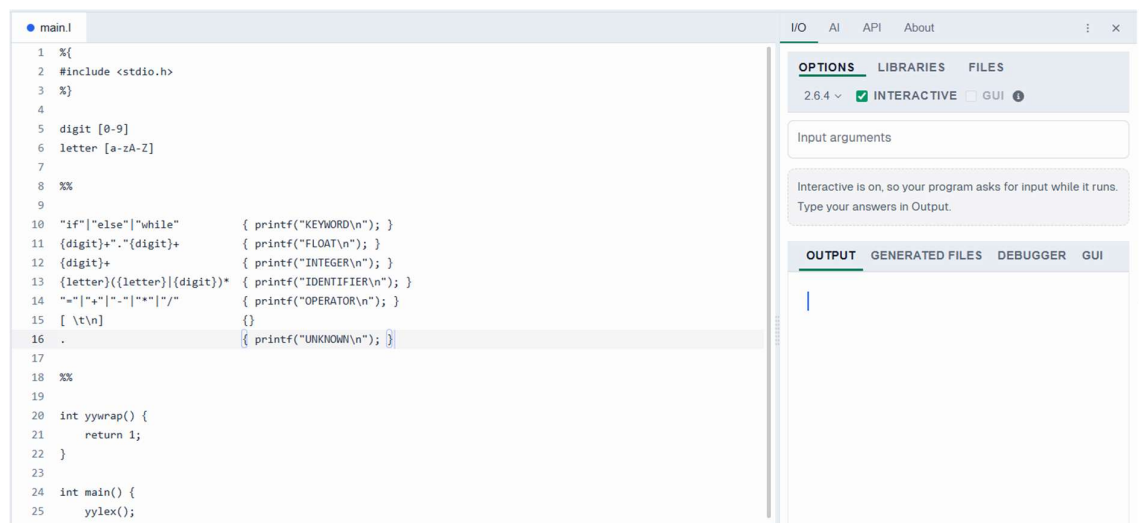
The screenshot shows the Online Flex Compiler interface. The code in `main.c` is as follows:

```
1  %{
2  #include <stdio.h>
3  %}
4
5  digit [0-9]
6  letter [a-zA-Z]
7
8  %%
9  {letter}({letter}|{digit})* { printf("IDENTIFIER\n"); }
10 "if"|"else"|"while"      { printf("KEYWORD\n"); }
11 {digit}+                 { printf("INTEGER\n"); }
12 {digit}+.{digit}+        { printf("FLOAT\n"); }
13 "={|"+|"-|"*|"/"        { printf("OPERATOR\n"); }
14 [ \t\n]                  { }
15 .                        { printf("UNKNOWN\n"); }
16
17 int main() {
18     yylex();
19     return 0;
20 }
21
```

The right-hand panel shows the **OUTPUT** tab with the following error messages:

```
jdoodle.1:10: warning, rule cannot be matched
jdoodle.1:17: warning, rule cannot be matched
jdoodle.1: In function 'yylex':
jdoodle.1:17:1: error: implicit declaration of fu
17 | int main() {
   | ^~~~~~
jdoodle.1:17:7: error: expected ';;' before '{' to
17 | int main() {
   |      ^~
   |      ;
```

Corrected Code:

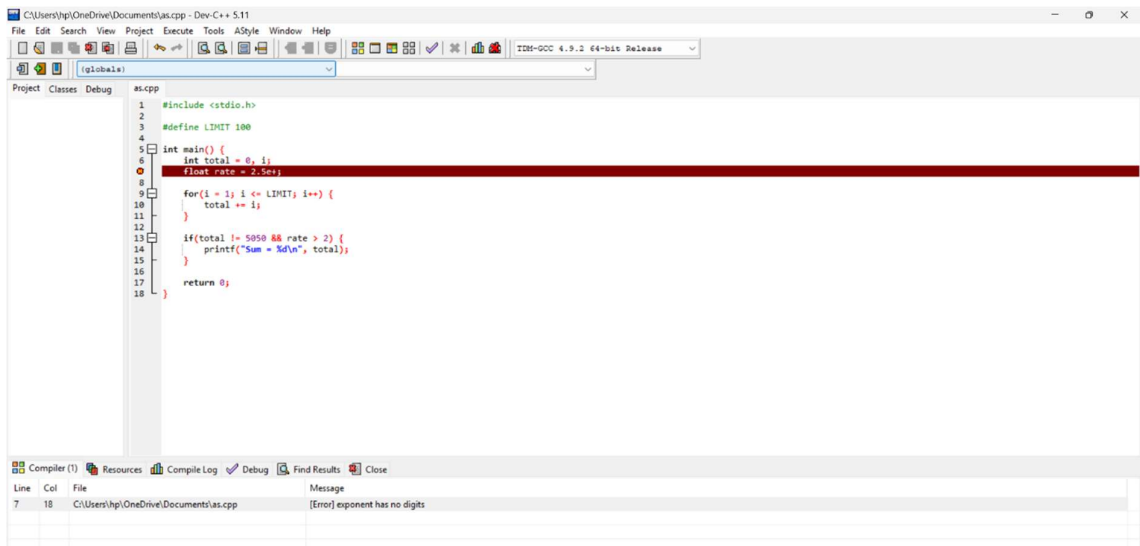


The screenshot shows the Online Flex Compiler interface with the corrected code in `main.c`:

```
1  %{
2  #include <stdio.h>
3  %}
4
5  digit [0-9]
6  letter [a-zA-Z]
7
8  %%
9
10 "if"|"else"|"while"      { printf("KEYWORD\n"); }
11 {digit}+                 { printf("INTEGER\n"); }
12 {digit}+.{digit}+        { printf("FLOAT\n"); }
13 {letter}({letter}|{digit})* { printf("IDENTIFIER\n"); }
14 "={|"+|"-|"*|"/"        { printf("OPERATOR\n"); }
15 [ \t\n]                  { }
16 .                        { printf("UNKNOWN\n"); }
17
18 %%
19
20 int yywrap() {
21     return 1;
22 }
23
24 int main() {
25     yylex();
26 }
```

The right-hand panel shows the **OUTPUT** tab, which is currently empty, indicating that the code has been successfully compiled without errors.

3.)Error Code:

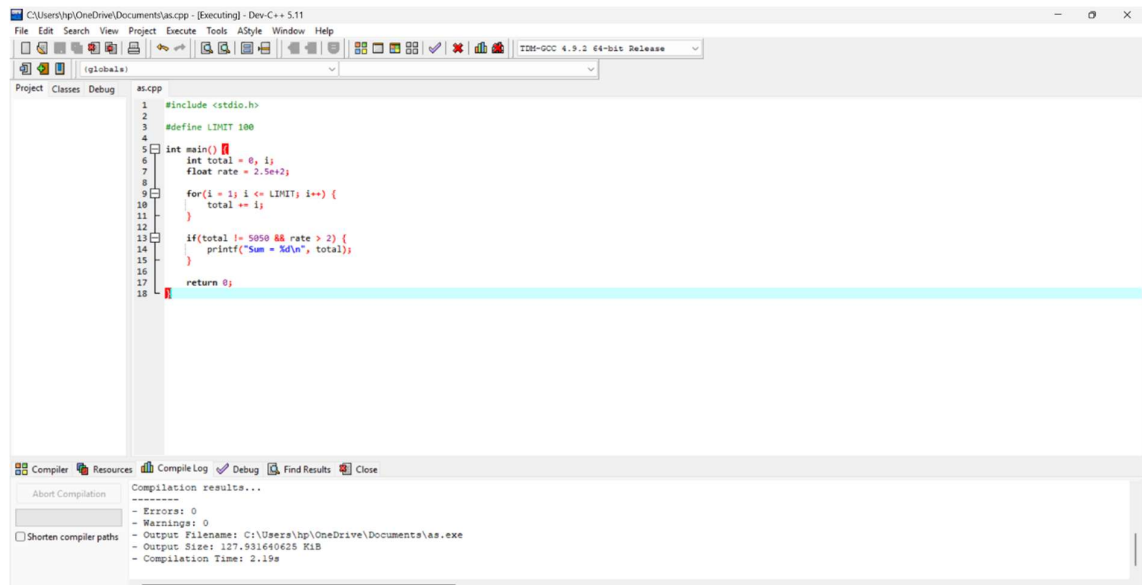


The screenshot shows the Dev-C++ IDE with a C++ file named `as.cpp`. The code is as follows:

```
1 #include <stdio.h>
2
3 #define LIMIT 100
4
5 int main() {
6     int total = 0, i;
7     float rate = 2.5e2;
8
9     for(i = 1; i <= LIMIT; i++) {
10         total += i;
11     }
12
13     if(total != 5050 && rate > 2) {
14         printf("Sum = %d\n", total);
15     }
16
17     return 0;
18 }
```

The error message in the bottom panel is: `[Error] exponent has no digits`, pointing to line 7, column 18, where the scientific notation `2.5e2` is used for the `rate` variable.

Corrected Code:



The screenshot shows the Dev-C++ IDE with the same C++ file `as.cpp`, but the `rate` variable is now assigned the integer value `250` instead of `2.5e2`. The code is as follows:

```
1 #include <stdio.h>
2
3 #define LIMIT 100
4
5 int main() {
6     int total = 0, i;
7     float rate = 250;
8
9     for(i = 1; i <= LIMIT; i++) {
10         total += i;
11     }
12
13     if(total != 5050 && rate > 2) {
14         printf("Sum = %d\n", total);
15     }
16
17     return 0;
18 }
```

The bottom panel shows the compilation results:

```
Compilation results...
-----
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\hpi\OneDrive\Documents\as.exe
- Output Size: 127,931,640,625 KiB
- Compilation Time: 2.19s
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand